

output plugins like stdout, InfluxDB, file and http. Additionally, Fluent Bit is easy to set up and requires no dependencies. So let us look at how to build and install it.

Building and installing Fluent Bit

Detailed information on the installation procedure can be found at <https://docs.fluentbit.io/manual/installation/sources/download-source-code>. Let us take a few short steps to build and install the project with default options.

Download the source code from git clone (<https://github.com/fluent/fluent-bit>).

Change to the build/directory

inside source:

```
cd build/
```

Let CMake configure the project, specifying where the root path is located:

```
cmake ../
```

Start compilation:

```
sudo make
```

To check if Fluent Bit has been installed, execute the command shown in Figure 1.

```
atibhi@atibhi:~$ whereis fluent-bit
fluent-bit: /usr/local/bin/fluent-bit /usr/local/etc/fluent-bit
```

Figure 1: Checking if Fluent Bit is installed

```
atibhi@atibhi:~/FBIT/fluent-bit$ fluent-bit -i dummy -o stdout
Fluent Bit v1.4.0
* Copyright (C) 2019-2020 The Fluent Bit Authors
* Copyright (C) 2015-2018 Treasure Data
* Fluent Bit is a CNCF sub-project under the umbrella of Fluentd
* https://fluentbit.io

[2020/05/07 00:31:59] [ info] [storage] version=1.0.3, initializing...
[2020/05/07 00:31:59] [ info] [storage] in-memory
[2020/05/07 00:31:59] [ info] [storage] normal synchronization mode, checksum disabled, max_chunks_up=128
[2020/05/07 00:31:59] [ info] [engine] started (pid=7089)
[2020/05/07 00:31:59] [ info] [sp] stream processor started
[0] dummy.0: [1508791720.00088756, {"message":"dummy"}]
[1] dummy.0: [1508791721.000598446, {"message":"dummy"}]
[2] dummy.0: [1508791722.000109263, {"message":"dummy"}]
[3] dummy.0: [1508791723.000330493, {"message":"dummy"}]
[0] dummy.0: [1508791724.000646374, {"message":"dummy"}]
[1] dummy.0: [1508791725.000570193, {"message":"dummy"}]
[2] dummy.0: [1508791726.000574700, {"message":"dummy"}]
[3] dummy.0: [1508791727.000129557, {"message":"dummy"}]
```

Figure 2: Getting started with a simple plugin. Sanity check to see if Fluent Bit is running correctly

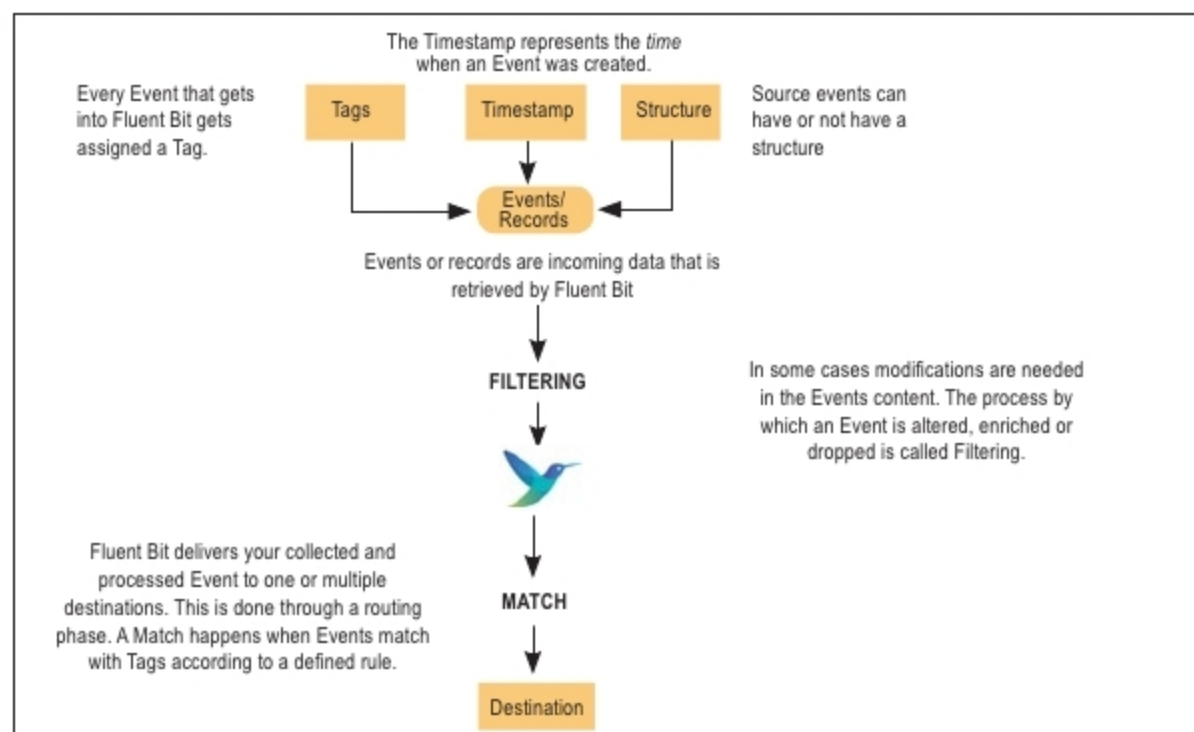


Figure 3: Flow chart explaining the key concepts of Fluent Bit

You can now use the dummy input plugin to check if Fluent Bit is working properly. Fluent Bit takes a dummy JSON record as input, and outputs it to standard output.

Now that you have Fluent Bit up and running, here are some key concepts that will help you to understand it better.

Key concepts

We hope the diagram in Figure 3 helps you get familiar with common Fluent Bit terminology. A detailed discussion about the key concepts is available at <https://docs.fluentbit.io/manual/concepts/key-concepts>. In the next section, we will discuss the data pipeline used by Fluent Bit, the concept of plugins and the configuration procedures.

Fluent Bit data pipeline

The efficient flow of data from one source to another is crucial for any data driven enterprise system. A data pipeline automates the flow of data and allows it to move easily from one station to the next. It automates the processes involved in retrieving, filtering, validating and loading data for further use. A data pipeline can be thought of as a necessary assembly line in today's world.

Fluent Bit defines a data pipeline that is cloud native and most of the stages in the pipeline are implemented through plugins. Figure 4 shows a schematic diagram for the data pipeline.

The first stage in the pipeline is the input stage, where Fluent Bit collects logs from different input sources using input plugins, which collect logs from syslog, standard input, kernel logs, etc. When an input plugin is created, an internal instance is also created with its own independent configuration.

Next comes the parsing stage. The events or records that come as input may be structured or unstructured. Structure defines a set of *keys* and *values* inside the event message.